

Universidad de San Carlos de Guatemala

Facultad de Ingeniería

Escuela de Ciencias y Sistemas

Laboratorio de Sistemas Operativos 1

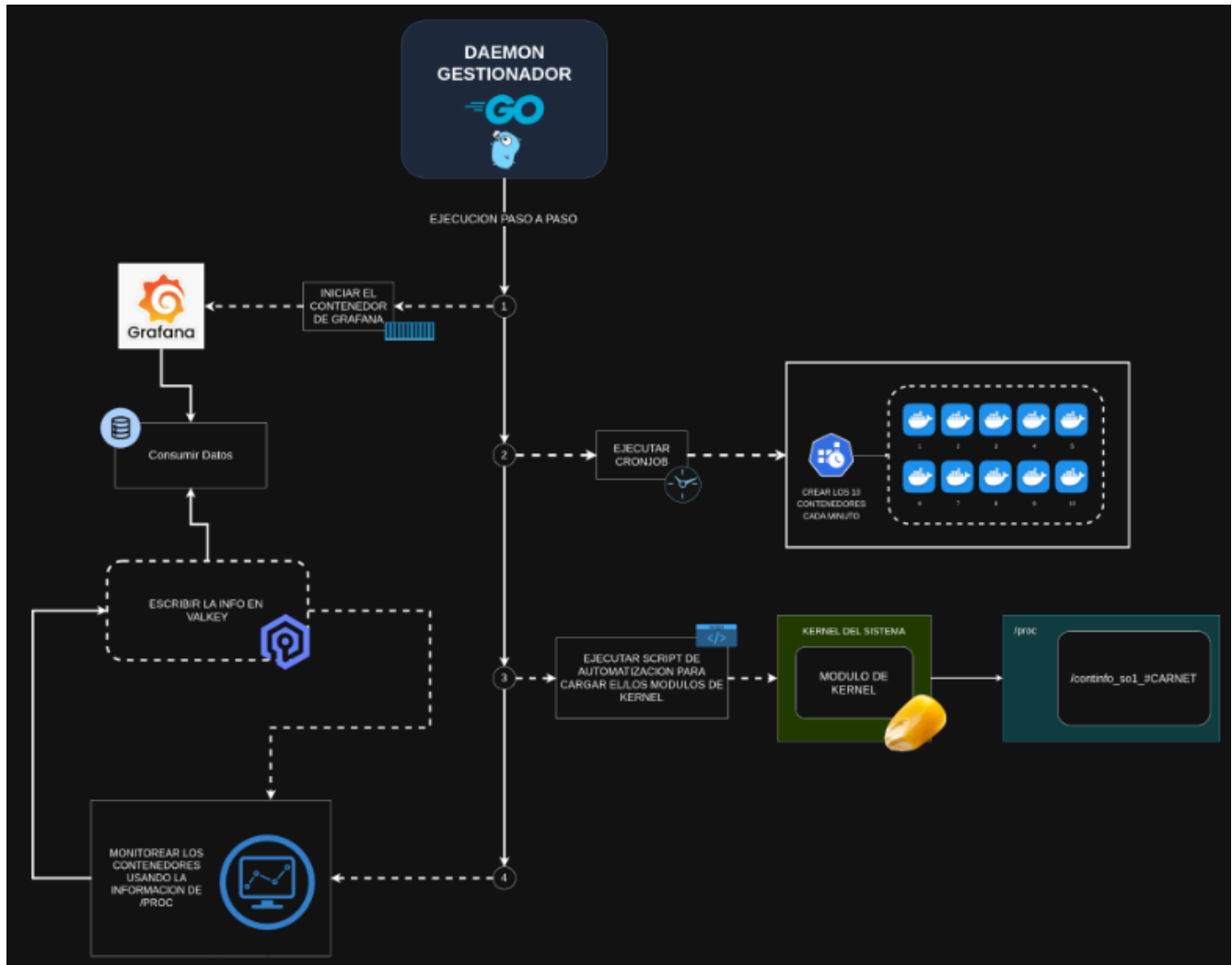
Manual Técnico y Guía de Instalación
Proyecto 2

Sebastián Gómez Lavarreda
201602929
Guatemala, marzo de 2026

Descripción General del Proyecto

El proyecto consiste en el desarrollo de un daemon en Go encargado de manejar los procesos relacionados a contenedores para mantener controlados los recursos del sistema. Este daemon se auxilia de un módulo de kernel el cual recopila la información de los procesos relacionados a los contenedores.

Se adjunta un diagrama de la distribución del sistema.



Descripción del módulo de kernel

El módulo de kernel fue desarrollado en C, utilizando las herramientas de desarrollo de Linux, por lo que es un requisito tenerlas instaladas para la ejecución del proyecto.

El módulo lee la información de todos los procesos del sistema y filtra por medio del nombre del task en la estructura de los procesos. Luego escribe los datos esenciales de uso de recursos por cada proceso en un archivo localizado en la carpeta /proc del sistema.

Descripción del Daemon de Go

El Daemon funciona levantando primero el contenedor de Grafana con su plugin de Redis, y el contenedor de Valkey que nos servirá como repositorio de datos para Grafana. Luego crea el cronjob que ejecutará los comandos para la creación de 5 contenedores aleatoriamente cada 2 minutos entre las imagenes recomendadas por los auxiliares. Finalmente, el daemon carga el módulo de kernel desarrollado para poder obtener la información de los procesos. Es importante mencionar que el módulo de kernel debe haber sido compilado antes de iniciar el daemon. Esto se detalla en la sección de Guía de Instalación.

El daemon queda ejecutando un ciclo “infinito” cada 20 segundos, que evaluará la cantidad de recursos disponibles y la utilización de recursos de cada contenedor, y tomará las decisiones de detener y eliminar un contenedor si lo considera necesario. Siempre respetando las restricciones impuestas en el enunciado, de mantener 3 contenedores de bajo consumo y 2 de alto consumo en todo momento.

En el ciclo, también se almacena la información de los contenedores en Valkey, permitiendo que Grafana genere reportes en tiempo real del funcionamiento del sistema.

Guía de Instalación

Requisitos

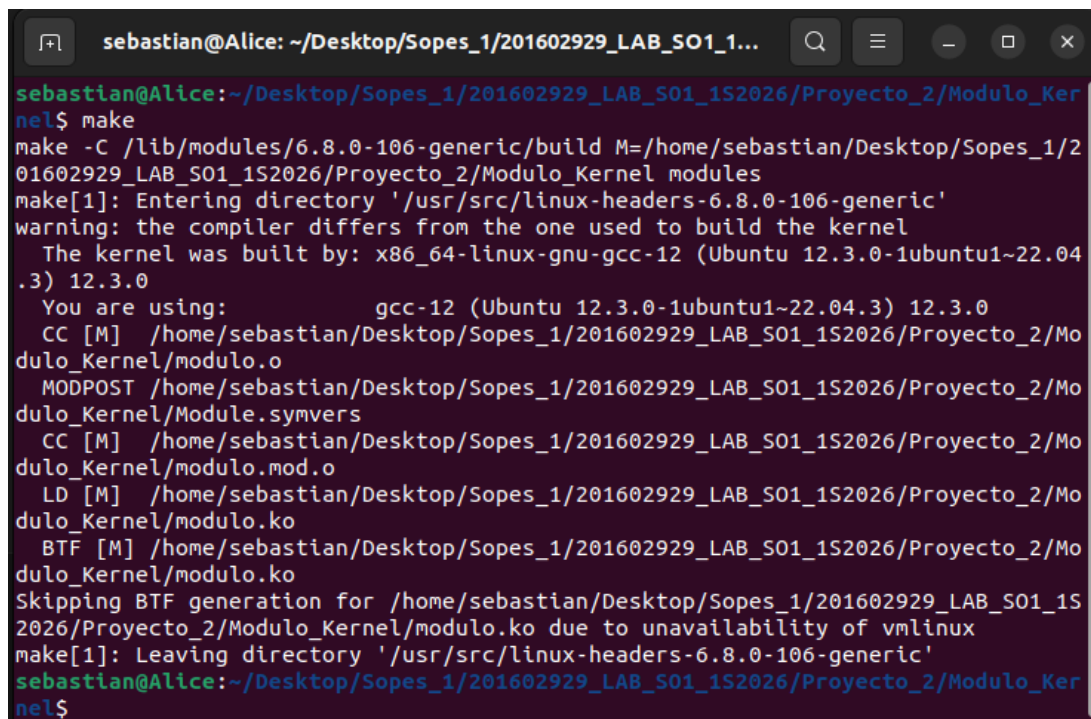
- Sistema operativo Linux
- Herramientas de compilación de módulos de Kernel de Linux.
- Herramientas de compilación de Go.
- Herramientas de compilación de módulos de Kernel de Linux con Rust (actividad extra).

Pasos para compilar el módulo de Kernel.

Debemos tener instaladas las herramientas de compilación de módulos de kernel de Linux.

```
sebastian@Alice: ~  
sebastian@Alice:~$ sudo apt install build-essential linux-headers-$(uname -r)  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
build-essential is already the newest version (12.9ubuntu3).  
linux-headers-6.8.0-106-generic is already the newest version (6.8.0-106.106~22.04.1).  
linux-headers-6.8.0-106-generic set to manually installed.  
The following packages were automatically installed and are no longer required:  
  arduino-builder arduino-core-avr arduino-ctags avr-libc avrdude binutils-avr  
  extra-xdg-menus gcc-avr java-wrappers libapache-pom-java libastylej-jni  
  libbatik-java libbcpg-java libbcprov-java libcommons-codec-java  
  libcommons-compress-java libcommons-exec-java libcommons-io-java  
  libcommons-lang3-java libcommons-logging-java libcommons-net-java  
  libcommons-parent-java libftdi1 libgoogle-gson-java libhidapi-libusb0  
  libhttpclient-java libhttpcore-java libjackson2-annotations-java  
  libjackson2-core-java libjackson2-databind-java libjaxp1.3-java  
  libjmdns-java libjna-java libjna-jni libjna-platform-java libjsch-java  
  libjssc-java libjzlib-java liblightcouch-java liblistserialsj-dev  
  liblistserialsj1 liblog4j2-java libmongodb-java librsyntaxtextarea-java  
  librxtx-java libsemver-java libserialport0 libslf4j-java libusb-0.1-4  
  libxalan2-java libxerces2-java libxml-commons-external-java  
  libxml-commons-resolver1.1-java libxmlgraphics-commons-java  
Use 'sudo apt autoremove' to remove them.
```

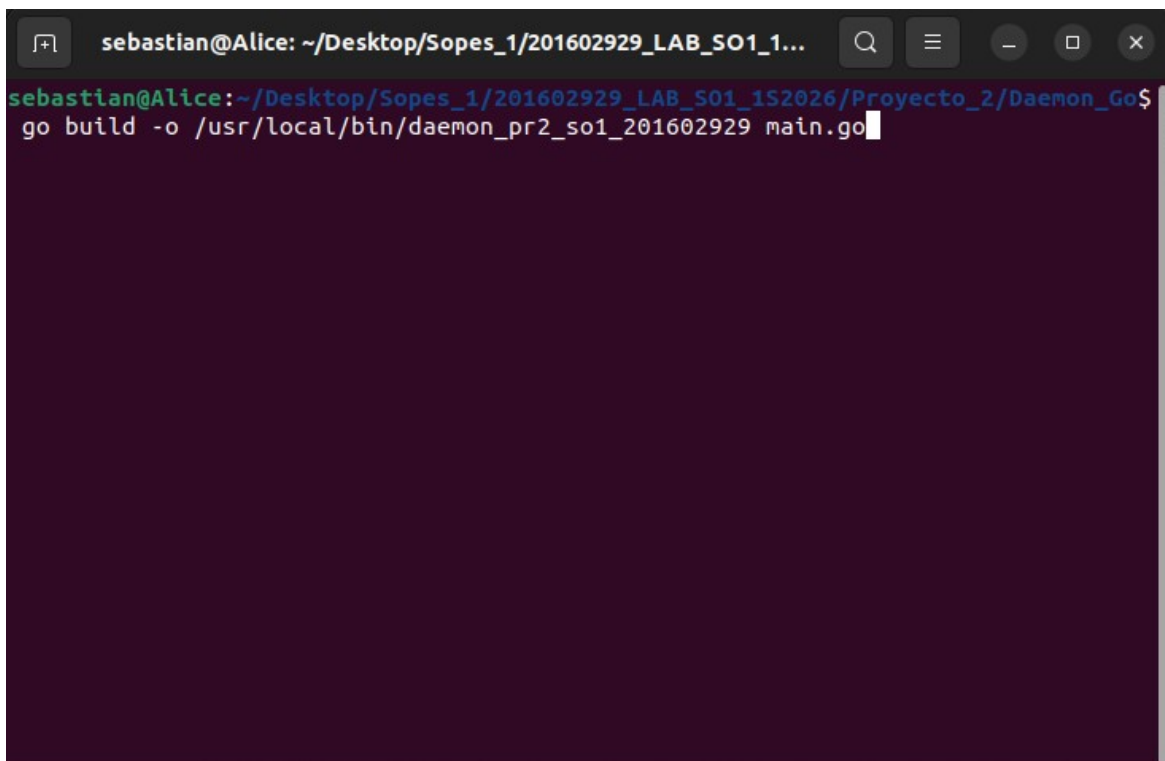
Navegamos a la carpeta Modulo_Kernel del proyecto. Ejecutamos el comando “make”.



```
sebastian@Alice: ~/Desktop/Sopes_1/201602929_LAB_SO1_1...
sebastian@Alice:~/Desktop/Sopes_1/201602929_LAB_SO1_1S2026/Proyecto_2/Modulo_Kernel$ make
make -C /lib/modules/6.8.0-106-generic/build M=/home/sebastian/Desktop/Sopes_1/201602929_LAB_SO1_1S2026/Proyecto_2/Modulo_Kernel modules
make[1]: Entering directory '/usr/src/linux-headers-6.8.0-106-generic'
warning: the compiler differs from the one used to build the kernel
The kernel was built by: x86_64-linux-gnu-gcc-12 (Ubuntu 12.3.0-1ubuntu1~22.04.3) 12.3.0
You are using: gcc-12 (Ubuntu 12.3.0-1ubuntu1~22.04.3) 12.3.0
CC [M] /home/sebastian/Desktop/Sopes_1/201602929_LAB_SO1_1S2026/Proyecto_2/Modulo_Kernel/modulo.o
MODPOST /home/sebastian/Desktop/Sopes_1/201602929_LAB_SO1_1S2026/Proyecto_2/Modulo_Kernel/Module.symvers
CC [M] /home/sebastian/Desktop/Sopes_1/201602929_LAB_SO1_1S2026/Proyecto_2/Modulo_Kernel/modulo.mod.o
LD [M] /home/sebastian/Desktop/Sopes_1/201602929_LAB_SO1_1S2026/Proyecto_2/Modulo_Kernel/modulo.ko
BTF [M] /home/sebastian/Desktop/Sopes_1/201602929_LAB_SO1_1S2026/Proyecto_2/Modulo_Kernel/modulo.ko
Skipping BTF generation for /home/sebastian/Desktop/Sopes_1/201602929_LAB_SO1_1S2026/Proyecto_2/Modulo_Kernel/modulo.ko due to unavailability of vmlinux
make[1]: Leaving directory '/usr/src/linux-headers-6.8.0-106-generic'
sebastian@Alice:~/Desktop/Sopes_1/201602929_LAB_SO1_1S2026/Proyecto_2/Modulo_Kernel$
```

Pasos para la compilar el Daemon

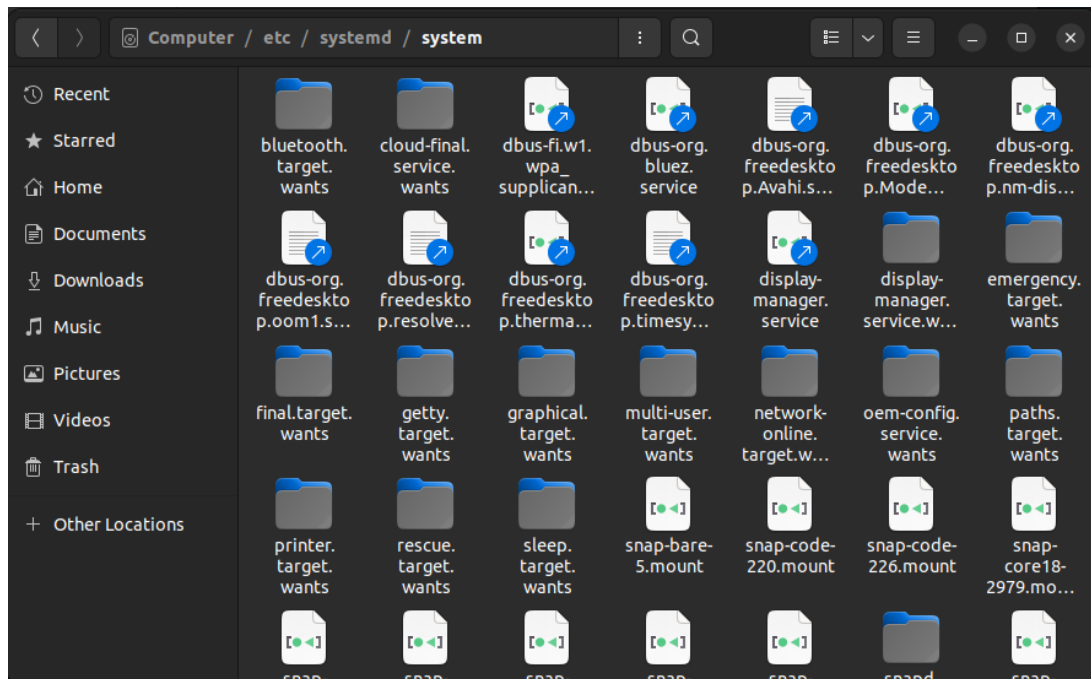
Navegamos a la carpeta Daemon_Go del proyecto. Ejecutamos el comando “go build -o /usr/local/bin/daemon_pr2_so1_201602929 main.go”.



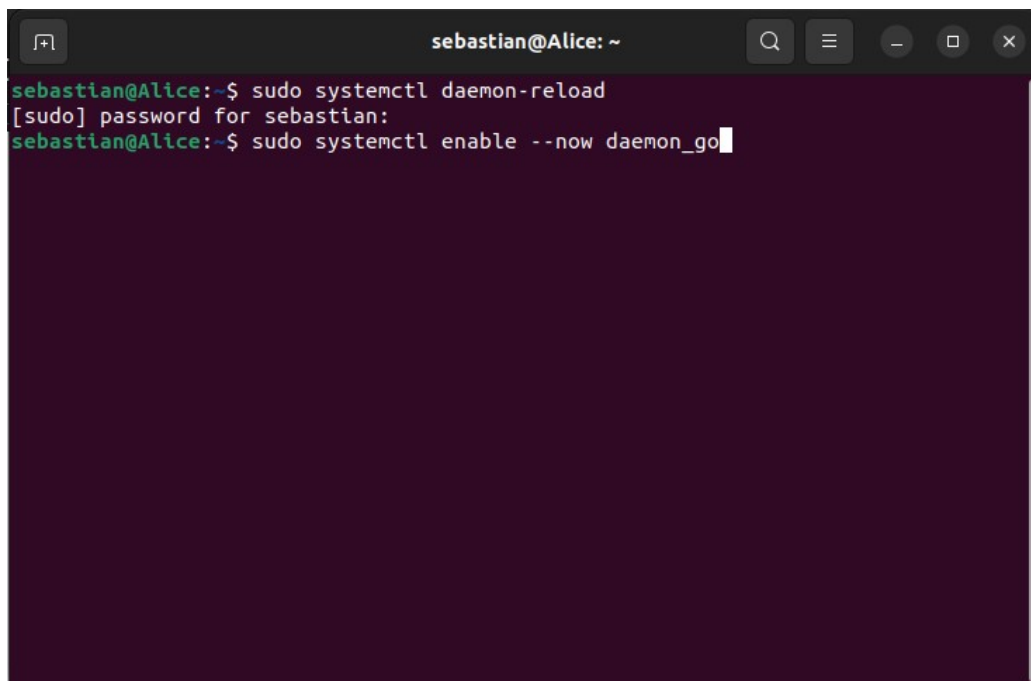
```
sebastian@Alice: ~/Desktop/Sopes_1/201602929_LAB_SO1_1...
sebastian@Alice:~/Desktop/Sopes_1/201602929_LAB_SO1_1S2026/Proyecto_2/Daemon_Go$ go build -o /usr/local/bin/daemon_pr2_so1_201602929 main.go
```

Pasos para la ejecución del Daemon

Copiamos el archivo `/Daemon_Go/daemon_go.service` de la carpeta del proyecto en la ruta `/etc/systemd/system/`.



Ejecutamos en una terminal el comando “`sudo systemctl daemon-reload`” para recargar el registro de servicios de nuestro sistema. Luego ejecutamos el comando “`sudo systemctl enable --now daemon_go`” para iniciar la ejecución de nuestro daemon.



Pasos para compilar el módulo de Kernel de Rust

Debemos tener instaladas las herramientas de compilación.

```
sebastian@Alice: ~  
sebastian@Alice:~$ sudo apt update  
[sudo] password for sebastian:  
Hit:1 https://download.docker.com/linux/ubuntu jammy InRelease  
Hit:2 http://security.ubuntu.com/ubuntu jammy-security InRelease  
Hit:3 http://gt.archive.ubuntu.com/ubuntu jammy InRelease  
Hit:4 http://gt.archive.ubuntu.com/ubuntu jammy-updates InRelease  
Hit:5 http://gt.archive.ubuntu.com/ubuntu jammy-backports InRelease  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
73 packages can be upgraded. Run 'apt list --upgradable' to see them.  
N: Skipping acquire of configured file 'stable/binary-i386/Packages' as repository 'https://download.docker.com/linux/ubuntu jammy InRelease' doesn't support architecture 'i386'  
sebastian@Alice:~$ sudo apt install build-essential flex bison libssl-dev libelf-dev llvm clang  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
bison is already the newest version (2:3.8.2+dfsg-1build1).  
build-essential is already the newest version (12.9ubuntu3).  
flex is already the newest version (2.6.4-8build2).  
libssl-dev is already the newest version (3.0.2-0ubuntu1.21).  
libssl-dev set to manually installed.
```

Instalamos Rust.

```
sebastian@Alice: ~  
sebastian@Alice:~$ curl --proto '=https' --tlsv1.2 https://sh.rustup.rs -sSf | sh  
info: downloading installer  
  
Welcome to Rust!  
  
This will download and install the official compiler for the Rust programming language, and its package manager, Cargo.  
  
Rustup metadata and toolchains will be installed into the Rustup home directory, located at:  
  
    /home/sebastian/.rustup  
  
This can be modified with the RUSTUP_HOME environment variable.  
  
The Cargo home directory is located at:  
  
    /home/sebastian/.cargo  
  
This can be modified with the CARGO_HOME environment variable.  
  
The cargo, rustc, rustup and other commands will be added to
```

Agregamos la ruta del ambiente y agregamos el componente de Rust “rust-src”.

```
sebastian@Alice: ~  
Rust is installed now. Great!  
  
To get started you may need to restart your current shell.  
This would reload your PATH environment variable to include  
Cargo's bin directory ($HOME/.cargo/bin).  
  
To configure your current shell, you need to source  
the corresponding env file under $HOME/.cargo.  
  
This is usually done by running one of the following (note the leading DOT):  
. "$HOME/.cargo/env"          # For sh/bash/zsh/ash/dash/pdksh  
source "$HOME/.cargo/env.fish" # For fish  
source "$($nu.home-path)/.cargo/env.nu" # For nushell  
sebastian@Alice:~$ source $HOME/.cargo/env  
sebastian@Alice:~$ rustc --version  
rustc 1.94.0 (4a4ef493e 2026-03-02)  
sebastian@Alice:~$ cargo --version  
cargo 1.94.0 (85eff7c80 2026-01-15)  
sebastian@Alice:~$ llvm-config --version  
14.0.0  
sebastian@Alice:~$ rustup component add rust-src  
info: downloading component 'rust-src'  
info: installing component 'rust-src'  
sebastian@Alice:~$
```

Si nuestro kernel no tiene soporte para compilar módulos de Rust, deberemos instalarlo. Podemos verificar con el comando “cat /boot/config-\$(uname -r) | grep CONFIG_RUST”. Si el comando no retorna ninguna información, debemos instalar el soporte para Rust.

```
sebastian@Alice: ~/Desktop/Sopes_1/201602929_LAB_SO1_1...  
sebastian@Alice:~/Desktop/Sopes_1/201602929_LAB_SO1_152026/Proyecto_2/Modulo_Rus  
t$ cat /boot/config-$(uname -r) | grep CONFIG_RUST  
sebastian@Alice:~/Desktop/Sopes_1/201602929_LAB_SO1_152026/Proyecto_2/Modulo_Rus  
t$ sudo apt install linux-headers-generic  
[sudo] password for sebastian:  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
The following additional packages will be installed:  
  linux-headers-5.15.0-171 linux-headers-5.15.0-171-generic  
The following NEW packages will be installed:  
  linux-headers-5.15.0-171 linux-headers-5.15.0-171-generic  
  linux-headers-generic  
0 upgraded, 3 newly installed, 0 to remove and 65 not upgraded.  
Need to get 15.2 MB of archives.  
After this operation, 103 MB of additional disk space will be used.  
Do you want to continue? [Y/n] y  
Get:1 http://gt.archive.ubuntu.com/ubuntu jammy-updates/main amd64 linux-headers  
-5.15.0-171 all 5.15.0-171.181 [12.4 MB]  
Get:2 http://gt.archive.ubuntu.com/ubuntu jammy-updates/main amd64 linux-headers  
-5.15.0-171-generic amd64 5.15.0-171.181 [2,849 kB]  
Get:3 http://gt.archive.ubuntu.com/ubuntu jammy-updates/main amd64 linux-headers  
-generic amd64 5.15.0.171.160 [2,332 B]  
Fetched 15.2 MB in 7s (2,325 kB/s)
```



```
sebastian@Alice: ~/Desktop/Sopes_1/201602929_LAB_SO1_1...
sebastian@Alice:~/Desktop/Sopes_1/201602929_LAB_SO1_152026/Proyecto_2/Modulo_Rus
t$ sudo apt install rustc cargo rust-src
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  libssh2-1 libstd-rust-1.75 libstd-rust-dev
Suggested packages:
  cargo-doc llvm-17 lld-17 clang-17
The following NEW packages will be installed:
  cargo libssh2-1 libstd-rust-1.75 libstd-rust-dev rust-src rustc
0 upgraded, 6 newly installed, 0 to remove and 65 not upgraded.
Need to get 118 MB of archives.
After this operation, 533 MB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://gt.archive.ubuntu.com/ubuntu jammy/universe amd64 libssh2-1 amd64 1
.10.0-3 [109 kB]
Get:2 http://gt.archive.ubuntu.com/ubuntu jammy-updates/main amd64 libstd-rust-1
.75 amd64 1.75.0+dfsg0ubuntu1~bpo0-0ubuntu0.22.04 [46.3 MB]
Get:3 http://gt.archive.ubuntu.com/ubuntu jammy-updates/main amd64 libstd-rust-d
ev amd64 1.75.0+dfsg0ubuntu1~bpo0-0ubuntu0.22.04 [41.6 MB]
Get:4 http://gt.archive.ubuntu.com/ubuntu jammy-updates/main amd64 rustc amd64 1
.75.0+dfsg0ubuntu1~bpo0-0ubuntu0.22.04 [3,404 kB]
Get:5 http://gt.archive.ubuntu.com/ubuntu jammy-updates/universe amd64 cargo amd
```

Si aun asi no podemos compilar nuestro modulo de Rust, es el kernel de nuestro SO no soporta la compilación de modulos con Rust. Realizaremos la prueba en KillrCoda.

Procedemos a compilar el módulo de kernel. Para ello, navegamos hasta la ruta que contiene el código de nuestro módulo de kernel y ejecutamos el comando “make”.